

Islamic University of Technology

Final Report

Course Code : EEE 4308

Course Name : Digital Electronics Lab

Project Name : SAP-1 Architecture-based 8-bit Computer Design and Implementation

Team : Byte_Masters

Name	Student ID
Irfan Hossain Rafi	230021331
Rafin Islam	230021323
Sanajid Arham Bakar	230021311
Raad al Hasan	230021322
Nazifa Noushin	230021117
Samiha Alam Purna	230021127

Supervisor:
Mr. Md. Rahib-Bin-Hossain

Junior Lecturer, Electrical & Electronic Engineering Department

Islamic University of Technology

Project Objective:

Objective: The overall goal of this project is to design and implement an 8-bit computer based on the SAP-1 (Simple-As-Possible) architecture using a combination of combinational and sequential digital circuits.

Brief Description of Project:

In this project we built an 8-bit SAP-1 in Proteus 8 professional software. Since it is 8-bit we utilized a single 8-bit bus for both address and data transfer for its subsequent operations. A 16x8 bit random access memory (RAM) is integrated for the storage of addresses and 8-bit data, enabling data manipulation and program execution. To manage program execution, the SAP-1 has a program counter that can effectively cycle through addresses from 0000 to 1111 (giving it access to the 16 memory locations). Furthermore, a manual input unit consisting of toggle switches is included to facilitate the initial writing of programs and data into the RAM. The memory address register (MAR) plays a crucial role and used to access data stored in the RAM.

For arithmetic and logic operations, the computer is equipped with an arithmetic and logic unit (ALU) capable of 8-bit addition and subtraction. It employs two registers, the Accumulator (register A) and register B, each with a capacity of 8 bits, for storing data necessary for ALU operations. To ensure the coordinated functioning of all components, instruction register and controller sequence is integrated, which synchronizes the activities of registers, memory, ALU, and other components.

Components:

- Clock Pulse
- Program Counter
- ALU
- Accumulator
- RAM
- MAR
- Address Selector
- B register
- Controller Sequencer
- Output register
- Instruction Register

Introduction to Different Modules and Components of SAP

1. Program Counter:

The Program Counter (PC) is a 4-bit binary counter in the SAP-1 that tracks the address of the next instruction to be fetched from RAM. It counts sequentially from 0000 to 1111 (decimal 0 to 15), allowing the system to access 16 memory locations. During each instruction cycle, the PC places its address on the Bus for the Memory Address Register (MAR) and then increments its value by 0001 to prepare for the next step in the program.

To manage execution flow, the PC uses a Count Enable signal to trigger increments and a Clear (CLR) signal to reset the count to 0000, ensuring the program can always restart from the first memory location. Like other SAP-1 modules, it is fully integrated with the bus for data transfer. If the counter reaches its limit of 1111, it wraps back to 0000 on the next clock pulse, a transition signalled by the Carry Out (CO) output.

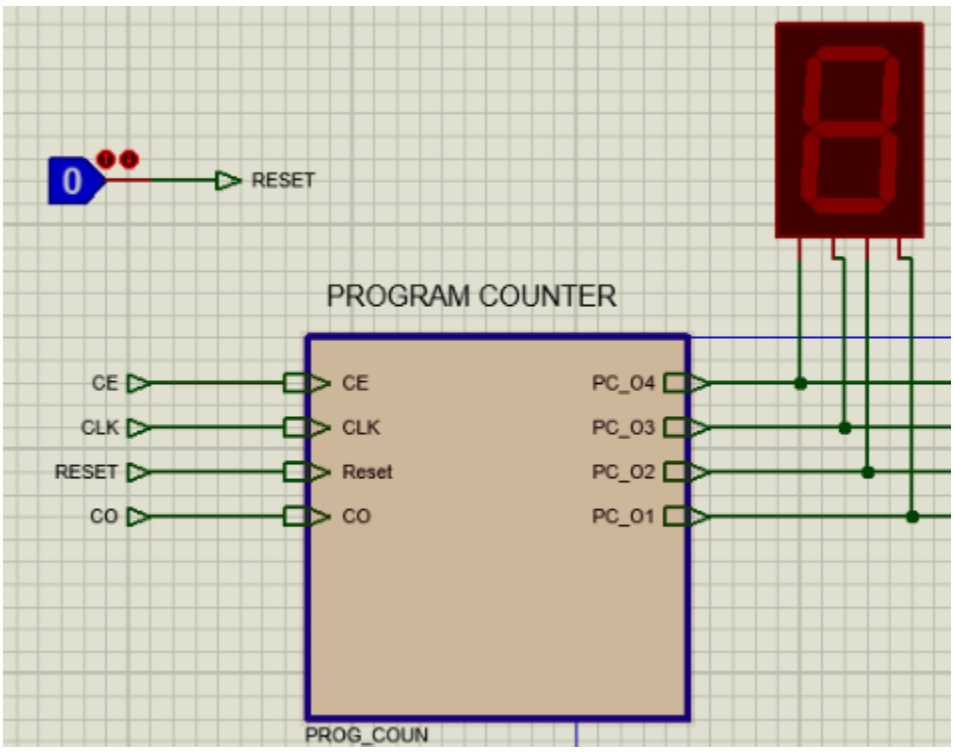


Figure: Block Diagram of the Program Counter Module (Parent Sheet)

Circuit Connection:

The Program Counter (PC) is implemented as a 4-bit synchronous counter using JK flip-flops (74HC107), where each flip-flop represents one bit of the address. All flip-flops share a common clock signal to ensure synchronized operation. The J and K inputs of each flip-flop are driven by combinational logic using AND gates (74HC08), such that each stage toggles based on the states of the lower-order bits, forming a binary counting sequence. A reset line, implemented through an inverter (74HC04), is connected to all flip-flops to asynchronously clear the counter to zero when required. The outputs of the flip-flops are connected to a tri-state buffer (74HC245), which interfaces the PC with the bus. Control signals such as Count Enable (CE) and buffer enable determine when the counter increments and when its contents are placed onto the bus.

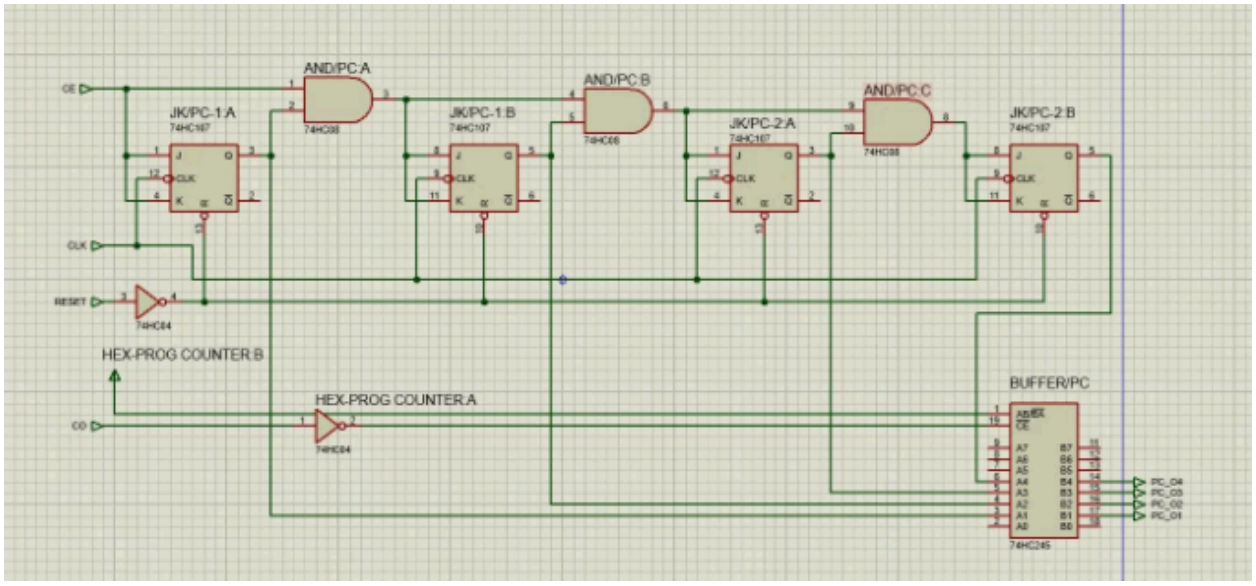


Figure: Circuit Implementation of the Program Counter (Child Sheet)

2. Clock Pulse Generator:

The Main Clock serves as the primary heartbeat of the computer, providing the driving signal necessary to synchronize all internal components. To allow for both high-speed execution and careful debugging, the system is designed with two distinct operational modes: Auto and Manual.

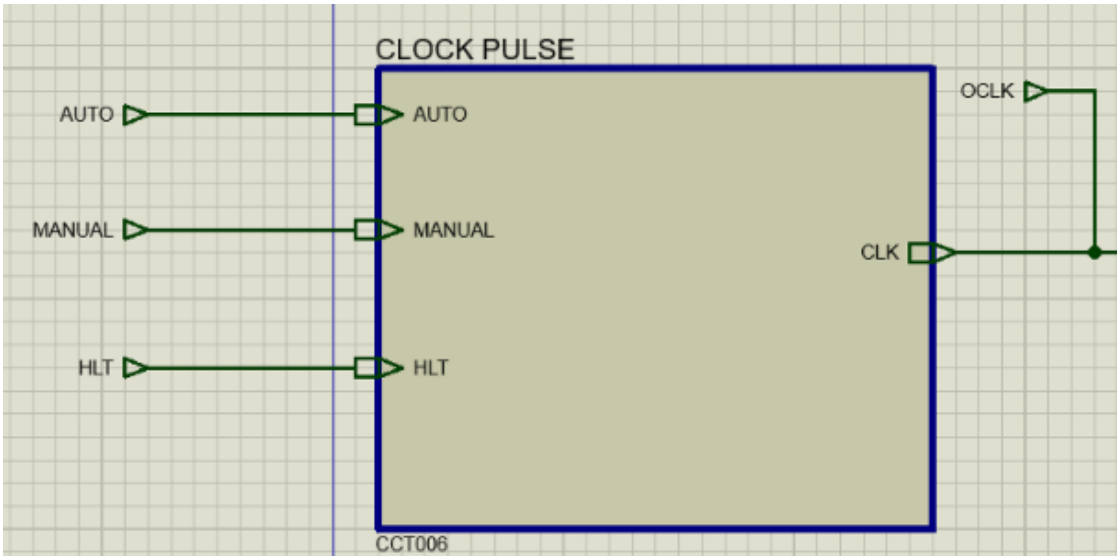


Figure: Clock Pulse Generator (Parent Sheet)

The module is constructed using a 555 Timer IC configured in an Astable Multivibrator circuit. This setup consists of several key components:

The 555 Timer IC: Functions as the central pulse generator, creating a continuous rectangular wave.

Passive RC Network: A combination of resistors and capacitors is used to define the charging and discharging cycles, which dictates the specific clock frequency or speed of the computer.

Logic Buffers/Inverters: These are utilized to clean the signal edges and provide both the standard CLK and inverted signals necessary for different synchronous components within the architecture.

The primary role of this module within the SAP-1 is to drive the Ring Counter (the Controller-Sequence unit), which acts as the system's conductor. This counter translates the raw clock pulses into a sequence of six distinct timing states, known as T1 through T6. During the first three states (T1, T2, T3), the clock facilitates the Fetch Cycle, moving the address from the Program Counter to the MAR and pulling the instruction from RAM. During the final three states (T4, T5, T6), the clock pulses enable

the Execution Cycle, where the specific operation such as adding a value to the Accumulator is carried out.

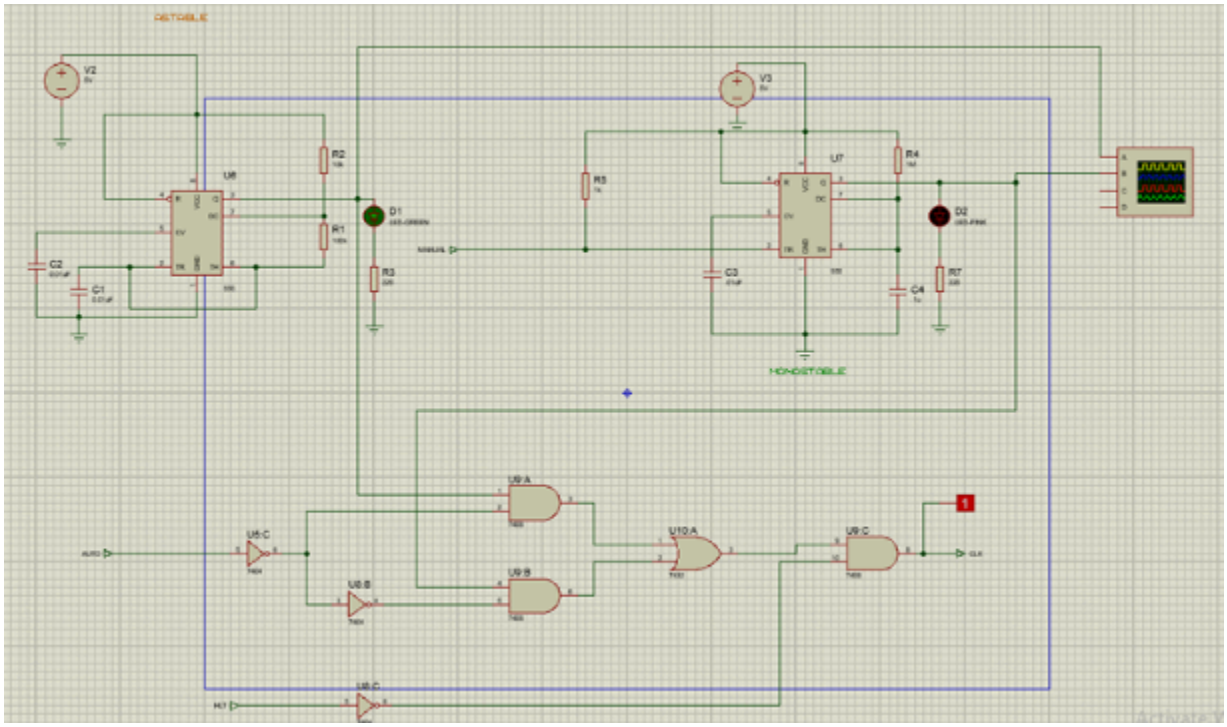


Figure: Child sheet of Clock Pulse Generator

Functionally, the clock pulses serve as the definitive trigger for all edge-sensitive components in the system. Most registers in this design, such as the Accumulator and the Instruction Register, are designed to latch or capture the data present on the W-bus specifically at the rising edge of the clock pulse. This means that even if data is fluctuating on the bus, it only becomes official and stored within a module at the exact moment the clock transitions from low to high. This edge-triggering mechanism is critical for the stability of the 8-bit memory system, as it prevents the registers from capturing noise or transitional data while the bus is still stabilizing.

Finally, the simulation allows for the manipulation of this clock signal to facilitate both high-speed execution and manual debugging. By adjusting the RC constant, often represented by a potentiometer in the Proteus schematic—the execution speed can be throttled down significantly. This slow-pulse mode is essential for observing the micro-operations in real-time, allowing for a visual audit of how control bits enable specific gates to move data between the RAM and the adder-subtractor.

3. MAR:

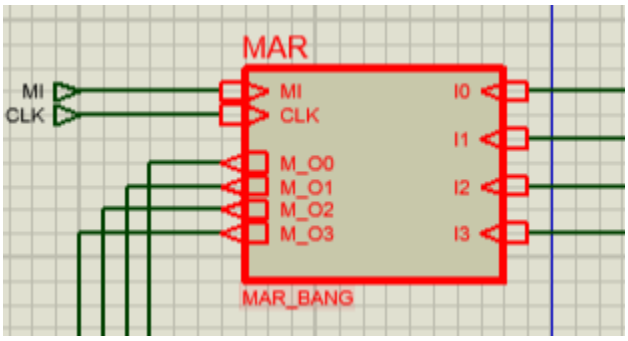


Figure: Parent Sheet of MAR

Parent Sheet:

The parent sheet shows the MAR block from the outside and how it connects the main system bus to the RAM.

Inputs: It gets a 4-bit data input through pins I0–I3, which are connected to the lower four bits of the main system bus.

Control Signals: The CLK pin sends the system clock to the control pin, and the MI (Memory Address Register In) control pin sends a load command.

Outputs: It sends a 4-bit address through pins M_O0–M_O3, which go straight to the address inputs of the RAM module.

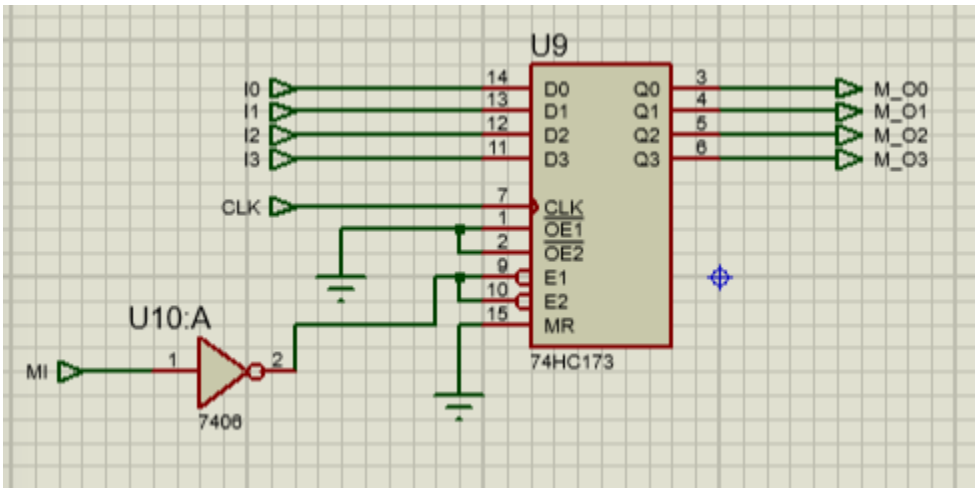


Fig: Child Sheet of MAR

Internal Components:

Memory Chip (U9): There is only one 74HC173 IC that keeps track of data. The SAP-1 only needs a 4-bit address bus, so this 4-bit D-type register is the right size.

Inverter (U10:A): A single 7406 NOT gate handles the incoming control signal.

Hardwired Logic: The Output Enable pins (OE1 and OE2) and the Master Reset pin (MR) of the register are all directly connected to ground. This means that the chip will always send out the data it has stored, and the hardware reset feature will always be off.

Function: The MAR stores memory addresses. The main 8-bit bus can't send the RAM its address directly because it is always getting new data and instructions. The MAR's job is to get a 4-bit address from the bottom half of the system bus at the right time and store it. This tells the RAM which of the 16 memory slots it should read from or write to.

Working Principle: When data comes in, a 4-bit address goes from the system bus to the MAR's input lines (I0–I3) and stays at the 74HC173 chip's data pins (D0–D3).

Load Command: When the computer's control unit decides it's time to grab this address, it sends a logical HIGH signal down the MI (Memory Address Register In) control line. The 7406 inverter (U10:A) changes this HIGH MI signal into a logical LOW. This active-LOW signal goes straight to the register's data enable pins (E1 and E2), which gets the register ready to receive data.

Data Capture: The 74HC173 chip immediately latches (saves) the 4 bits that are waiting at its inputs when the system clock (CLK) goes up a level.

Continuous Output: The output enable pins are connected directly to ground, so the new address quickly goes out through the Q0–Q3 pins to the M_O0–M_O3 lines. The RAM gets this address right away and keeps it there, locked and steady, until the next MI command is given

4. RAM:

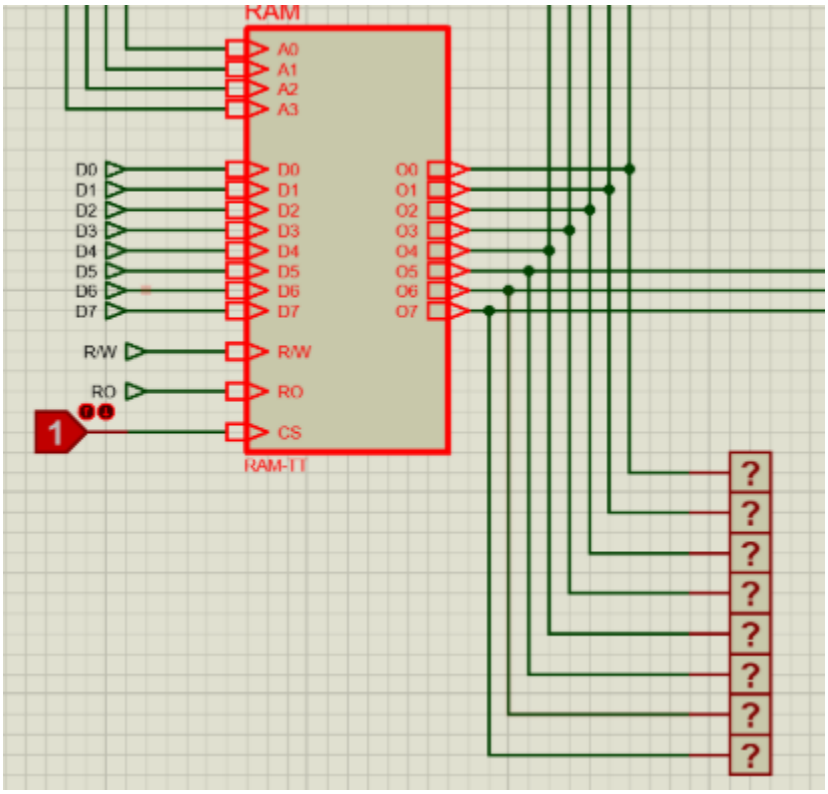


Figure: Parent sheet of RAM

Parent Sheet: The parent sheet shows how the RAM block connects to the rest of the SAP-1 computer from the outside.

Inputs: It gets a 4-bit address through pins A0–A3, which means it can find up to 16 specific memory slots. It also gets 8-bit data through pins D0 to D7.

Outputs: It sends 8-bit data through pins O0 to O7.

R/W (Read/Write), RO (RAM Out), and CS (Chip Select) are the control signals that tell it what to do.

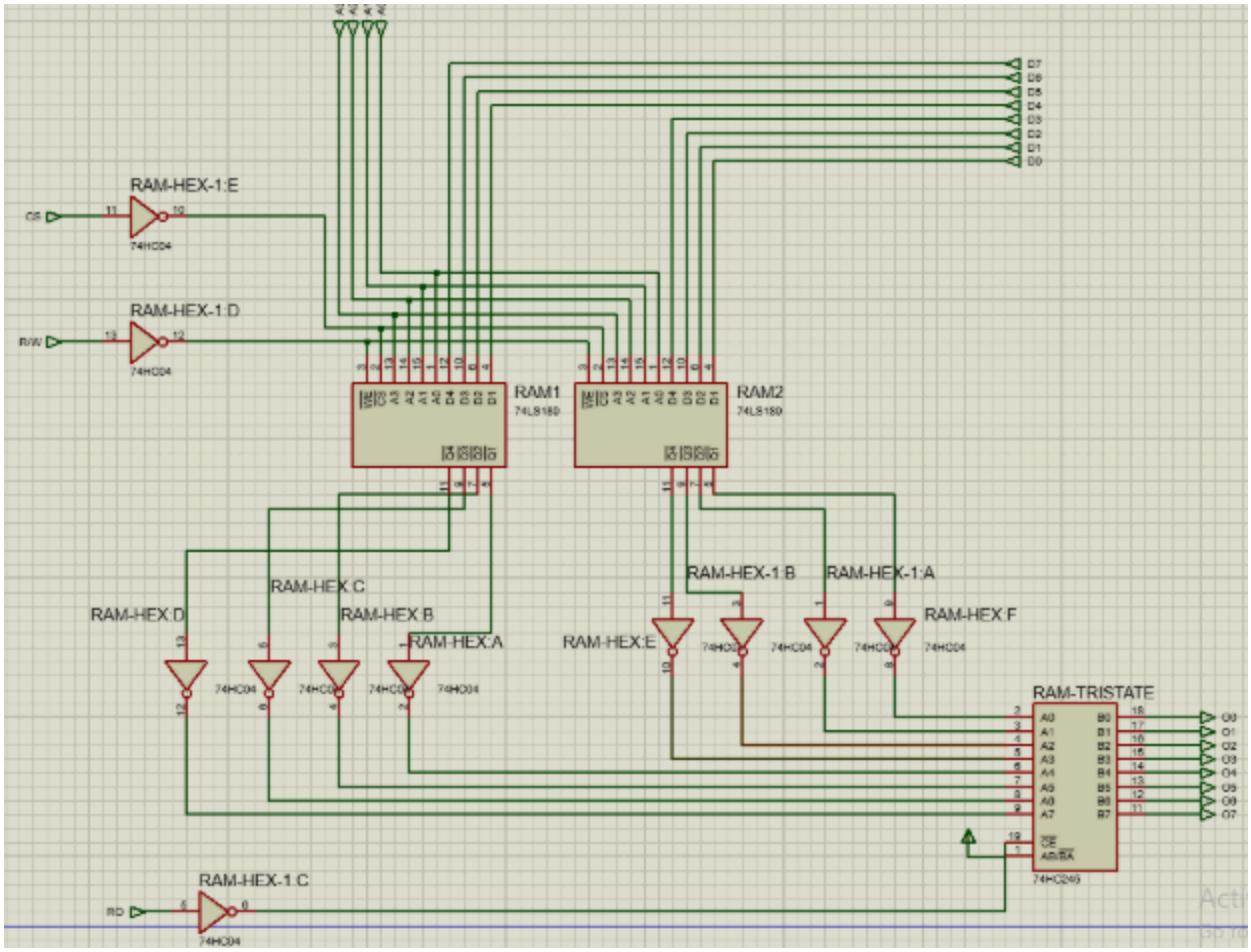


Fig: Child Sheet of RAM

Internal Components:

Memory Chips (RAM1 and RAM2): The two 74LS189 ICs are still the main storage. They are all 16x4-bit RAM chips, and together they can handle the full 8-bit data width (4 bits each).

Hex Inverters (74HC04): A lot of NOT gates are used in this. Some of them (like RAM-HEX-1:D, E, and C) are used to flip incoming control signals (R/W, CS, and RO). The other RAM-HEX chips (A-D and 1:A, B, E, F) are used to turn the data that comes out of the RAM chips back to normal.

This uses a single 74HC245 octal bus transceiver instead of the two 74LS240s that were used before. It is the gatekeeper, deciding when the RAM's output can go to the computer's 8-bit bus.

Function: The main function is still the same. It is the main volatile memory for the SAP-1 computer and can hold up to 16 bytes of program instructions and calculation data. It gets a 4-bit address to find a certain byte, and it can either save an 8-bit word to that address or read an 8-bit word from it.

Working Principle: Addressing: A 4-bit address (A0–A3) comes in and connects directly to the address pins of both 74LS189 chips at the same time.

Saving Data: The 8-bit data waits at inputs D0 to D7. The R/W line sends a signal to the computer when it wants to save data. This signal goes through the 74HC04 inverter (RAM-HEX-1:D) and turns on the Write Enable (WE) pins on both RAM chips, which writes the data to the chosen address. The Chip Select should also be turned on through the inverter.

Reading Data and Correction: The 74LS189 chips get the data when they output it, but they naturally spit it out upside down (inverted). This raw data goes straight to the bank of 74HC04 inverters, which changes all 8 bits back to their correct 1s and 0s.

Bus Output: The corrected data is now at the inputs of the 74HC245 Tri-State buffer. The computer must send the RO (RAM Out) command before it can go through. When RO is triggered (and RAM-HEX-1:C inverts it to turn on the CE pin), the buffer opens its gates, and the last 8 bits of data flow out of pins O0-O7 onto the system bus.

5. Address Selector

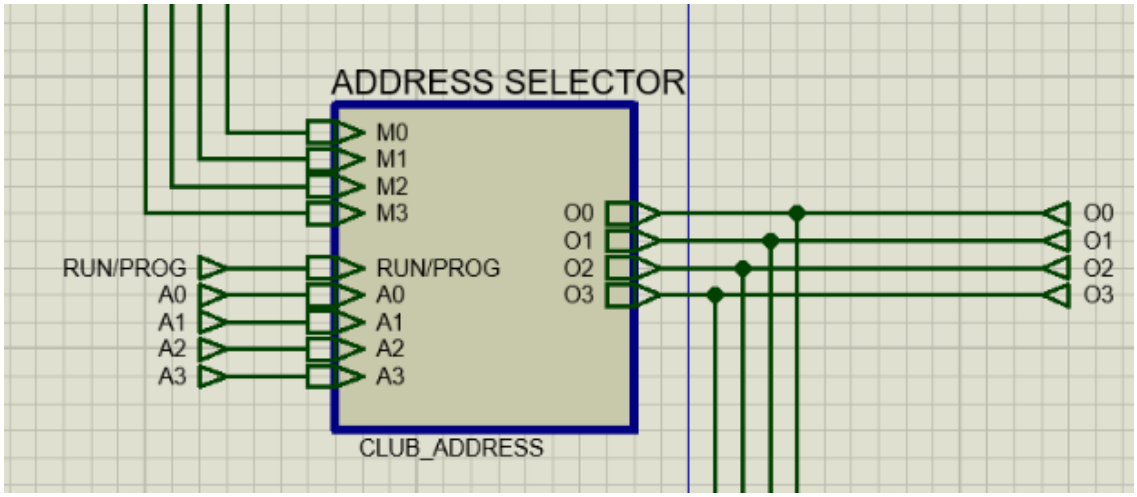


Figure: Address selector parent sheet

Components Used:

Core Logic IC: One 74LS157, which is a Quad 2-to-1 line data selector/multiplexer.

Inputs:

A0-A3: 4-bit address lines coming from the Program Counter (PC).

M0-M3: 4-bit manual address lines coming from user-controlled dip switches.

RUN/PROG: 1-bit control line used to select the operating mode.

Outputs: O0-O3: 4-bit output address lines that feed into the Memory (RAM) address inputs.

Function:

The primary function of the Address Selector is to act as a traffic director for the computer's memory address lines. Because the SAP-1 memory needs to receive addresses from two different sources at different times (from the user during programming, and from the Program Counter during execution), this module ensures only one address source is connected to the RAM at any given moment. It toggles the computer between its "Programming" state and its "Execution/Run" state.

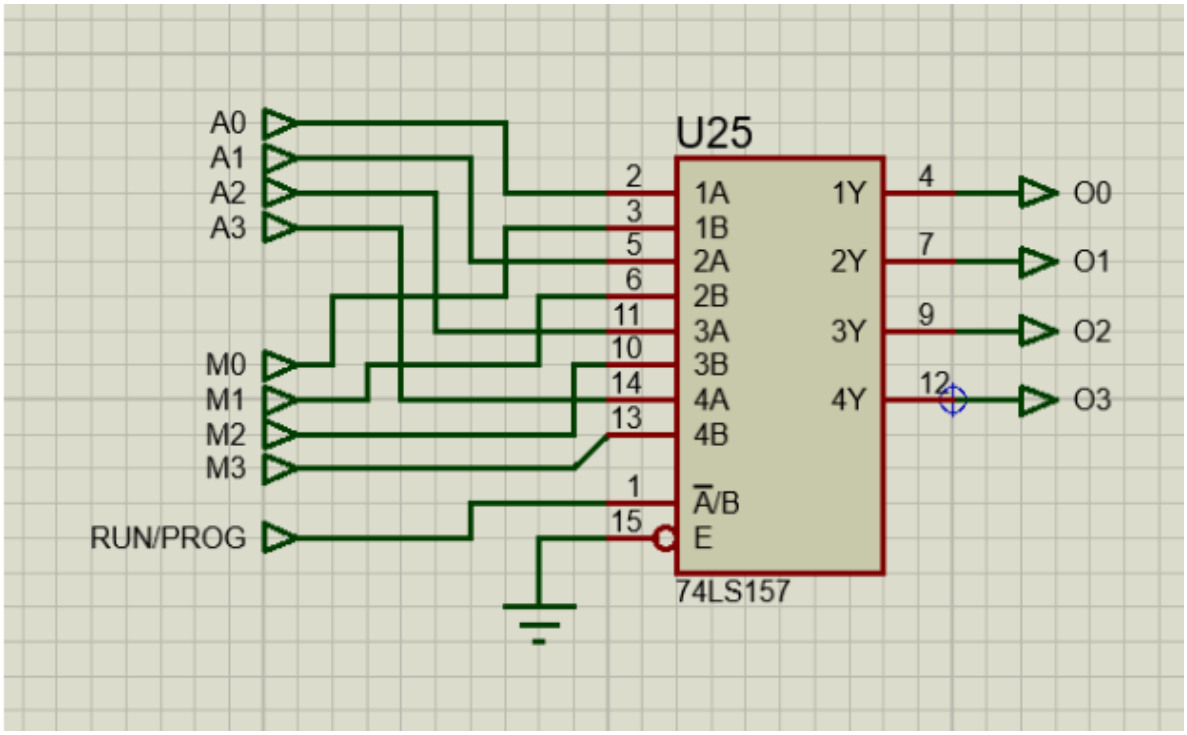


Fig: Address Selector child sheet

Working principle:

Multiplexing: The 74LS157 takes in two sets of 4-bit data. The 'A' inputs (Pins 2, 5, 11, 14) are wired to the Program Counter (A0-A3), and the 'B' inputs (Pins 3, 6, 10, 13) are wired to the manual input switches (M0-M3).

Mode Selection: The RUN/PROG line is connected to the select pin (~A/B, Pin 1) of the IC. This pin determines which set of inputs gets passed to the outputs.

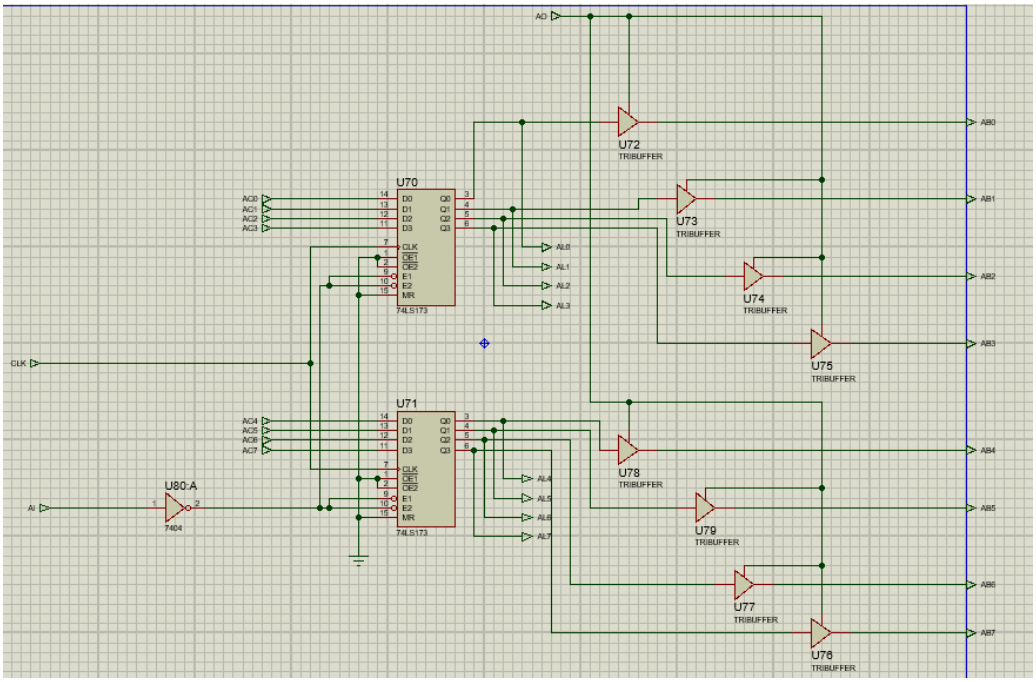
Run Mode: When the RUN/PROG signal is LOW (0), the IC selects the 'A' inputs. The 4-bit address from the Program Counter is routed to the outputs (O0-O3). This allows the SAP-1 to automatically fetch the next instruction from memory during the standard fetch cycle.

Program Mode: When the RUN/PROG signal is HIGH (1), the IC selects the 'B' inputs. The 4-bit address set by the user's manual switches is routed to the outputs. This allows the user to target specific memory addresses to manually load instructions and data before running a program.

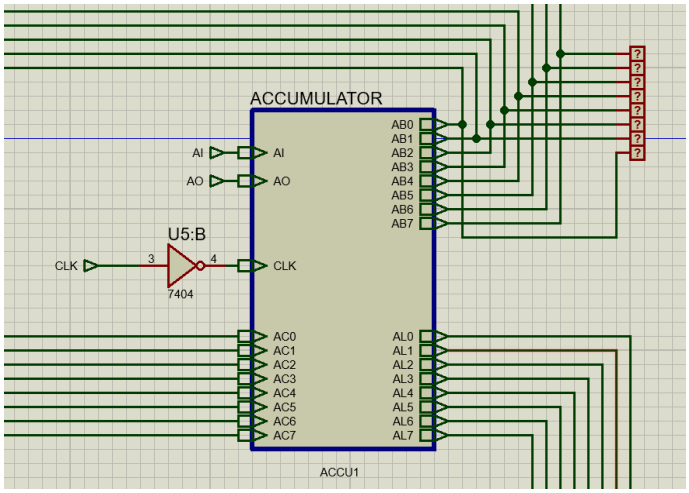
Continuous Operation: The Enable pin (~E, Pin 15) is an active-low input and is permanently tied to ground. This means the chip is always enabled and is constantly outputting the selected 4-bit address to the memory module.

6. Accumulator: It is also known as A register, as it is the first register that stores the values from the bus. It is made of two 74LS273 chips. Though we made it with 74HC273, we changed it afterwards as per the suggestions. We used tristate buffers to control by AO pin when it returns to the bus.

The accumulator works with an inverted clock pulse. There is another pin, the accumulator input. It only works when AI pin is enabled. Otherwise, it will stay off. Accumulator takes the data from the bus and feeds it to the ALU, which helps us to sum or subtract a number. It needs to be mentioned that there is no pin for A register to ALU. It will automatically feed data to the ALU from the accumulator.



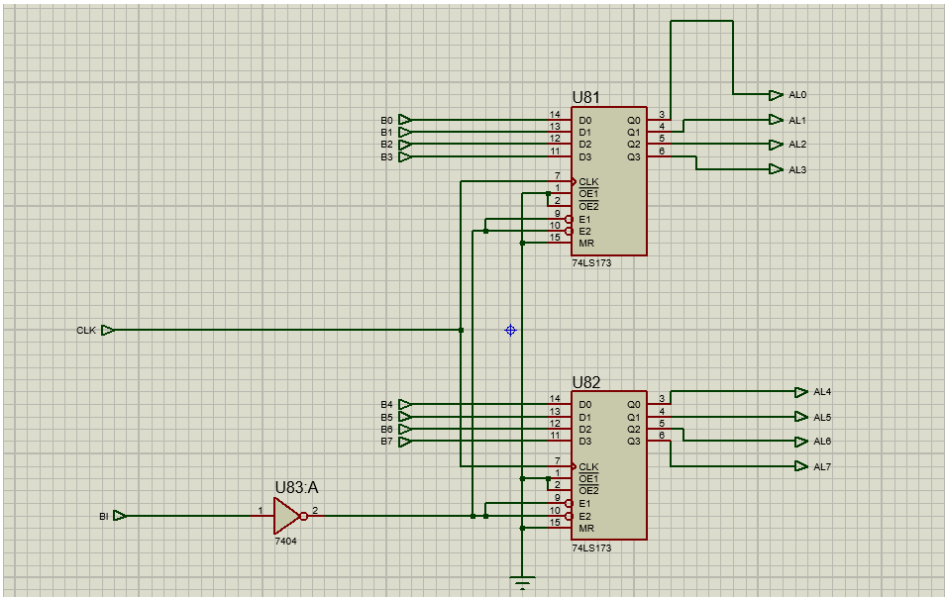
Schematic: Childsheet



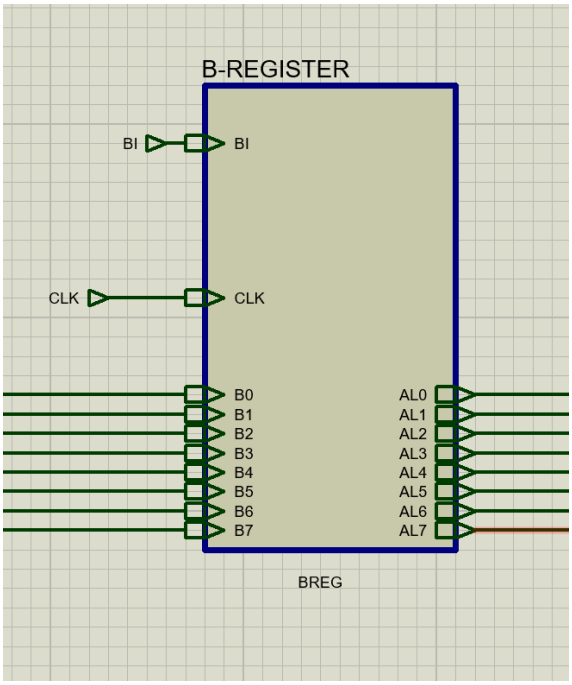
Schematic: Parentsheet

7. B-register: The working principle of the B register is quite easier than the A register, as it takes an 8-bit input from the bus and feeds it to the ALU. It is the second number for sum or subtraction. It is also made of two 74LS273 chips. It is easier to use chips rather than flipflops. Flipflops are time-consuming to make and not feasible for use.

B register has an enable pin and a clock pulse in. The clock pulse won't be inverted like the previous one. The output only goes to ALU. We won't see any tristate buffer in this module.



Schematic: Childsheet



Schematic: Parentsheet

8. Arithmetic Logic Unit (ALU):

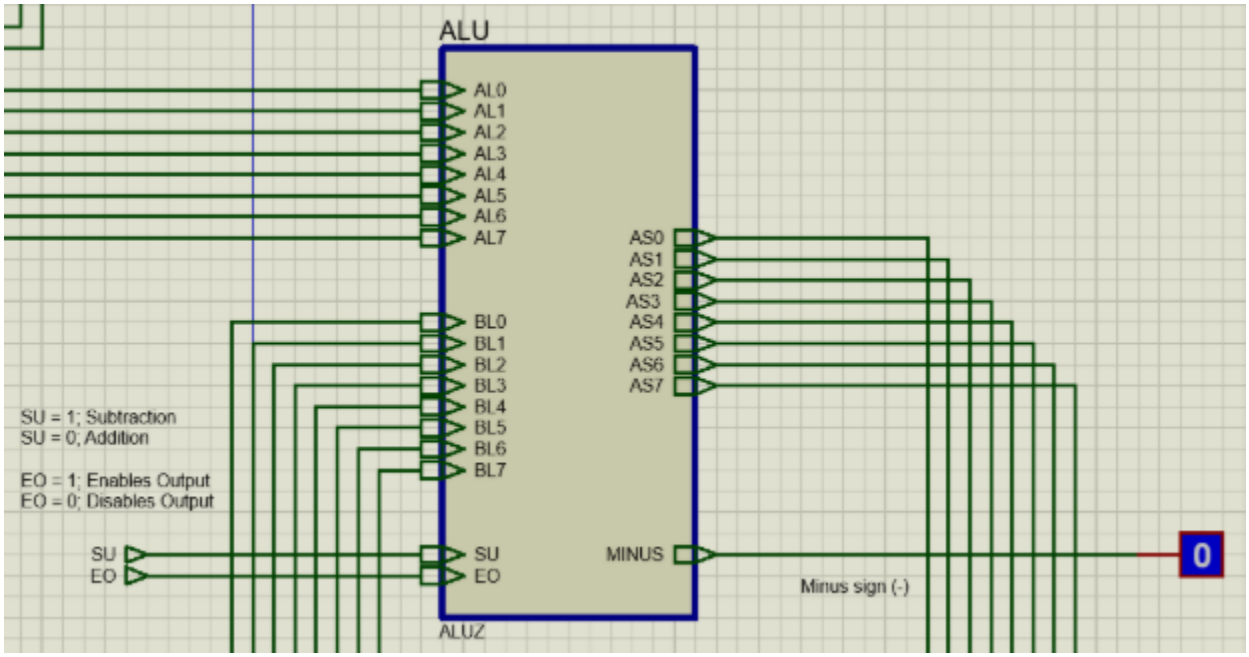


Fig: ALU Parent sheet

Components Used:

- Arithmetic Core: Sixteen 1-bit Adders, cascaded and arranged into two distinct 8-bit columns (Stage 1 on the right, Stage 2 on the left).
- Inversion Logic: 74LS86 (Quad 2-input XOR gates) placed at the B-register inputs and between the two adder stages.
- Output Control: Tri-state buffers (likely 74LS126 or similar active-high buffers) on the final outputs.
- Inputs:
    - AL0–AL7: 8-bit data from the Accumulator.
    - BL0–BL7: 8-bit data from the B Register.
    - SU: 1-bit Subtraction control line (1 = Subtract, 0 = Add).
    - EO: 1-bit Enable Output control line (1 = Enabled, 0 = Disabled).
- Outputs:
    - AS0–AS7: 8-bit Result Output (wired to the W-bus).
    - MINUS: 1-bit indicator light to show negative results.

Function:

The ALU is the mathematical brain of the computer. Its primary function is to perform addition and subtraction on the data stored in the Accumulator and the B Register. However, this specific custom ALU does more than the standard SAP-1 design: instead of outputting negative numbers in difficult-to-read two's complement form, it dynamically converts negative results back into standard natural binary (absolute magnitude) and flags it with a dedicated "Minus" indicator.

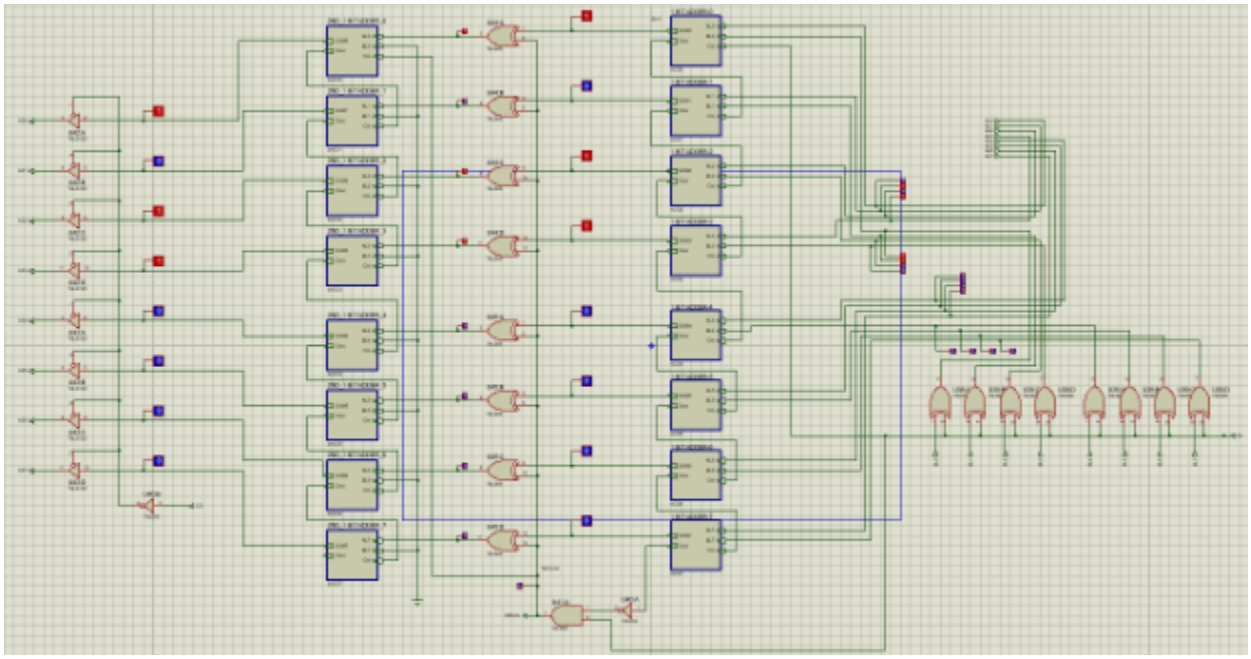


Fig: ALU child sheet

Working Principle:

The signal flow moves from right to left across the schematic, processing the math in several distinct stages:

1. Input and 1's Complement Stage (Right):
   When the control unit sends a command, the 8 bits from the Accumulator (AL) and the B Register (BL) enter the ALU. The B-inputs first pass through a bank of XOR gates.
   - During Addition (SU = 0), these XOR gates act as pass-throughs, leaving the B data unchanged.

- During Subtraction (SU = 1), the XOR gates act as inverters, flipping all the B bits (creating a 1's complement). The SU line also feeds into the Carry-In of the first adder, effectively adding 1. This completes the 2's complement conversion of the B register (A + ~B + 1), turning the addition circuit into a subtractor.

2. First Arithmetic Stage (Right Column):
   The first column of eight 1-bit adders computes the initial result. If the operation was addition, or if it was a subtraction resulting in a positive number, this output is the final correct magnitude. However, if A - B results in a negative number, this column outputs the negative number in two's complement.
3. Magnitude Conversion Stage (Middle XORs & Left Column):
   To get the natural binary representation, the circuit must detect a negative result (via the sign bit/carry out) and negate the two's complement number.
   - If positive: The middle XOR gates pass the data unchanged, and the second column of adders simply adds 0.
   - If negative: The negative sign bit triggers the MINUS output indicator. It also triggers the middle bank of XOR gates to invert the output of the first stage, and forces the second column of adders to add 1. This performs a second 2's complement operation on the data, converting the negative binary string back into its readable, natural absolute magnitude.
4. Output Stage (Far Left):
   The final natural binary result sits at the tri-state buffers waiting for permission to leave the module. When the Control Unit drives the EO (Enable Output) line HIGH (1), the buffers open, and the calculated answer (AS0-AS7) is placed onto the W-bus, typically to be stored back into the Accumulator. When EO is LOW (0), the outputs remain in a high-impedance state, preventing interference with other modules on the bus.

9. Instruction Register:

In SAP-1 architecture the 'Instruction Register' works as a temporary storage which is being performed and acts as a linkage between RAM and CU.

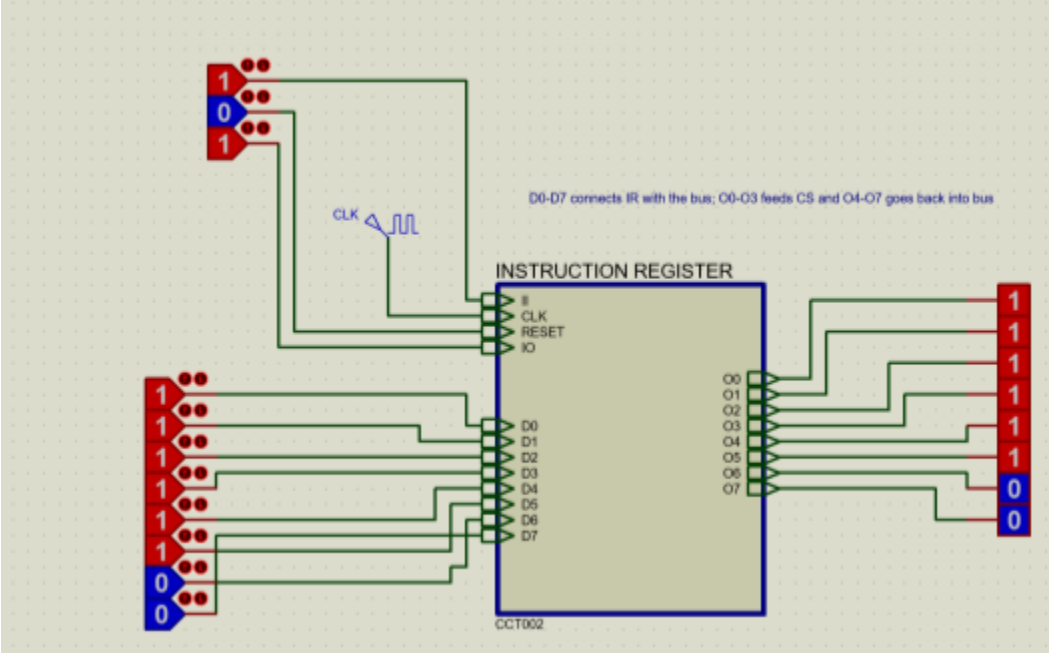


Figure: Instruction Register

The primary role of the Instruction Register is to receive an instruction from the RAM during the Fetch Cycle and hold it while the computer understands and performs this instruction.

Unlike other registers that hold data to be used for arithmetic e.g Accumulator and addresses e.g. the MAR, the IR holds a command that instructs it what to do next.

Functions:

The Instruction Register (IR) is where the command processing occurs in SAP-1. During the fetch portion of the cycle, the instruction is brought in from the RAM to the bus. The IR generates its load line to capture and store the 8 bit word in its instruction register. The IR is responsible for holding the instruction stable while the computer processes it so that the instruction does not disappear from the bus during processing.

The IR separates the 8 bit command instruction into two 4 bit commands for completely different purposes. The high 4 bits (O0 - O4) are called the Op Code (Operation Code) and they are connected to the Controller and will never return to the Main Data Bus. The Controller will read the Op Code and use this information to decide what to do next (add, subtract, load, etc.) and then generate control signals needed to complete the selected task.

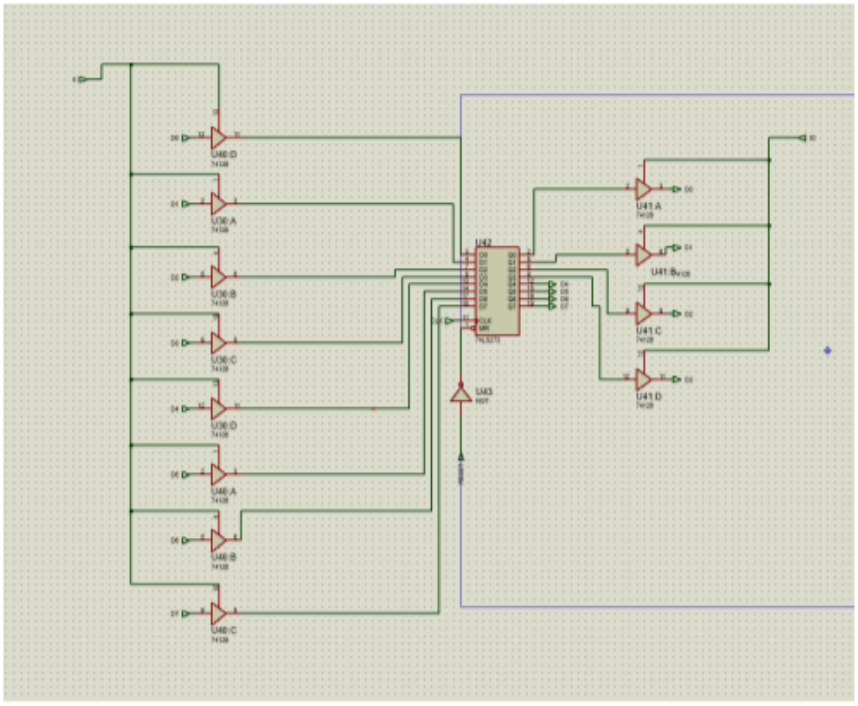
The low 4 bits (4-7) are called the Operand and they most generally hold a Memory Address. The Operand works with the Main System Bus while the Op Code does not.

Working Principle:

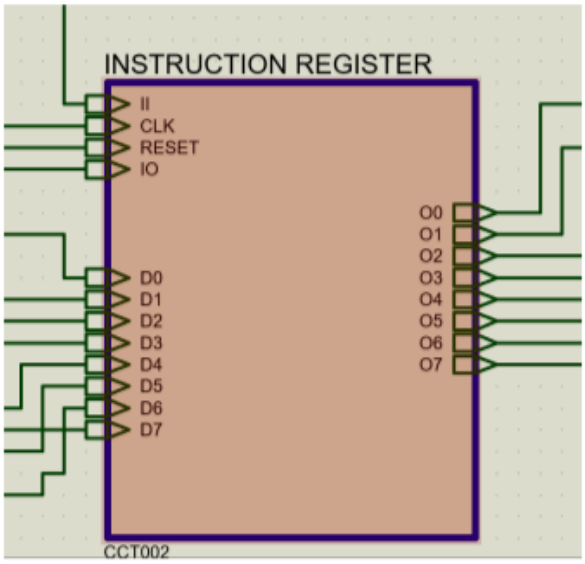
One of the important functions of the Instruction Register (IR) in SAP-1 is to act as a temporary storage and a means of allocation centre for the instructions during the clock cycle. In this context, an Instruction Register works by capturing an 8-bit instruction from the system bus and dividing that single instruction into two functional 4-bit nibbles (the Operand and Opcode). By splitting the instruction into its two functional parts, the Control Unit can determine precisely what function needs to be executed, and the remaining portions of the machine can locate the necessary data to accomplish that execution.

The IR begins its primary function during the third state of the fetch cycle, also known as State T3. At this point in time, the Program Counter has identified the memory address to be used, and the corresponding 8-bit instruction has been placed on the bus by the RAM. The Controller generates a low LI (Load Instruction) signal that indicates to the IR to latch the currently available 8-bit word from the bus. Once the signal has gone back to high, the instruction is locked into the IR, remaining stable and available for use even though the data on the bus continues to change.

Once the 8-bit instruction has been stored in the IR, the internal circuitry of the IR treats the 8-bit instruction as two distinct parts when decoding and executing.



Child Sheet of Instruction Register

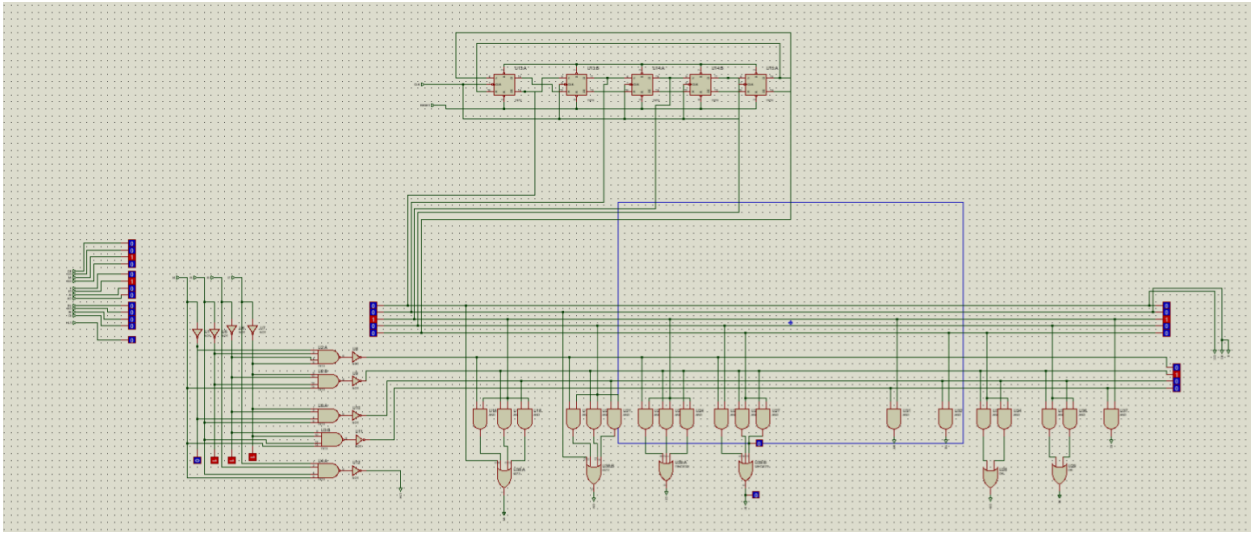


Parent Sheet of Instruction Register

10. Controller:

The Controller of the SAP-1 architecture is the central processing unit. The Instruction Register, which is synced with a ring counter that measures T-state time intervals from T1 to T5, provides the 4-bit opcode that the system decodes.

In order to produce output signals that will activate other system components utilizing their "Enable" and "Load" activation capabilities, the controller processes several inputs to produce twelve different control signals throughout its clock pulse operation. Depending on the command, the computer can go through its Fetch cycle and Execution cycle since data transmission onto and from the bus occurs at precisely the proper time.



Working Principle:

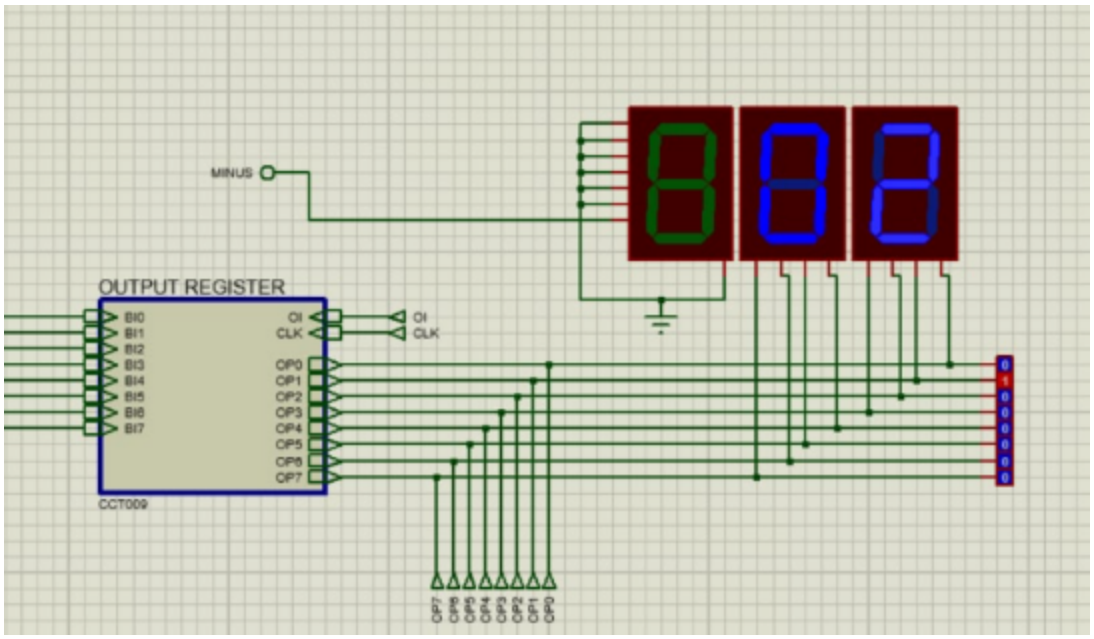
By producing control signals based on the current instruction and a timing counter, the controller in the SAP-1 architecture functions as the central nervous system, precisely directing the timing and sequencing of data transfer.

It divides the machine cycle into six different states by combining the 4-bit opcode from the Instruction Register with timing signals (T1 through T65) produced by a ring counter. In order to execute the "Fetch" cycle and transfer the instruction from memory to the Instruction Register, the controller outputs a predetermined set of signals during the first three states (T1–T2). The controller's logic circuit decodes the

particular opcode during the execution states and activates the special set of control lines required for the instruction to be executed smoothly.

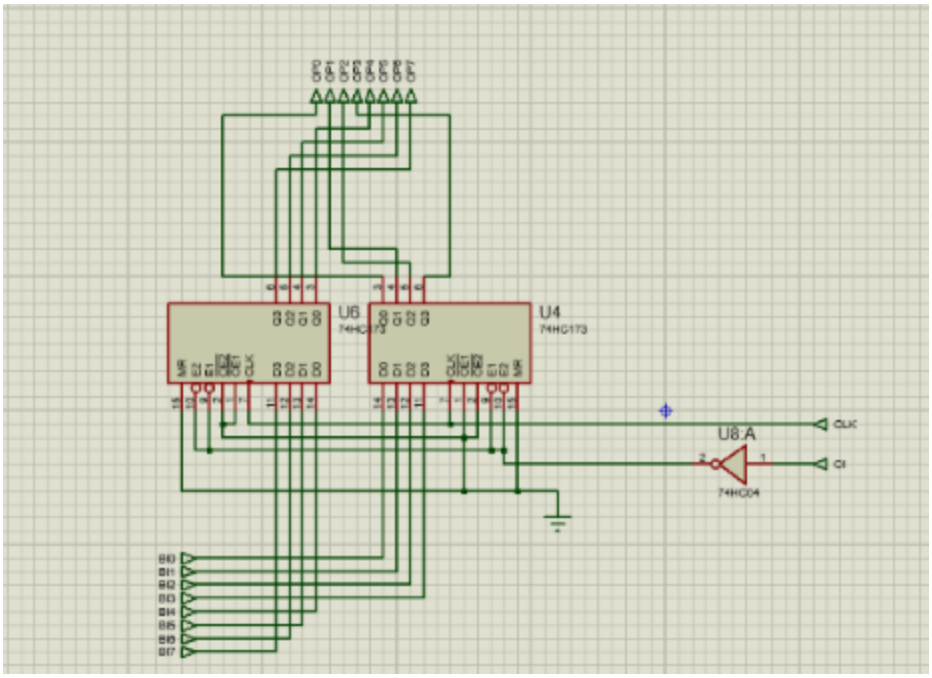
11. Output Register:

It is the final stage of the SAP-1 architecture, responsible for capturing processed data and presenting it in a human-readable format.



The module is primarily constructed using two 74HC173 (or 74LS173) 4-bit D-type registers cascaded to form an 8-bit storage unit.

- Data Inputs (BI0–BI7): These pins are connected directly to the common 8-bit bus to receive data from the accumulator or other modules.
- Control Logic: The register features a Load signal labeled as OI in the circuit which is routed through a 74HC04 Inverter. This ensures that the register only captures data when the Controller-Sequence unit specifically pulls the Output Enable line low.
- Display Interface: The outputs (OP0–OP7) are wired to a decoding circuit, which in this simulation consists of 7-segment displays to show the final numerical result.



The functional operation of the Output Register in the SAP-1 architecture is defined by its role as a synchronized bridge between the internal bus and the external display. Its primary duty is to latch or capture the 8-bit binary data from the bus specifically during an OUT instruction. This process is governed by the Controller-Sequence unit, which activates the OI signal during the relevant timing state (usually T4). When this signal is pulled low and a rising clock edge occurs, the registers (U4 and U6) snapshot the voltage levels on the BI0–BI7 lines, transitioning the data from the volatile bus into semi-permanent storage.

Once the data is captured, the Output Register provides electrical isolation and stability. Because the SAP-1 uses a shared bus, the data on the bus changes rapidly as the computer fetches new instructions or performs intermediate arithmetic in the Adder-Subtractor. Without the Output Register's latching capability, the 7-segment displays would flicker uncontrollably, reflecting every transient bit move. Instead, the 74HC173 chips maintain their output state (OP0–OP7) regardless of subsequent bus activity, ensuring the user sees a steady, consistent result until the next explicit OUT command is executed.

In the final functional stage, the module translates these stored binary bits into a human-readable format. The 8-bit output is fed into a decoder ,often a BCD (Binary

Coded Decimal) or Hex-to-7-segment driver which controls the individual segments of the display units. In the Proteus simulation provided, this allows the binary value of the Accumulator to be visualized as a numerical value.

12. Bus Integration:

The System Bus in SAP-1 is a shared 8-bit communication pathway that connects all modules of the computer, allowing them to exchange data in a coordinated manner. Instead of using dedicated connections between each pair of components, all modules are connected to this common bus and operate in a time-shared manner, where only one module is allowed to place data onto the bus at any given time.

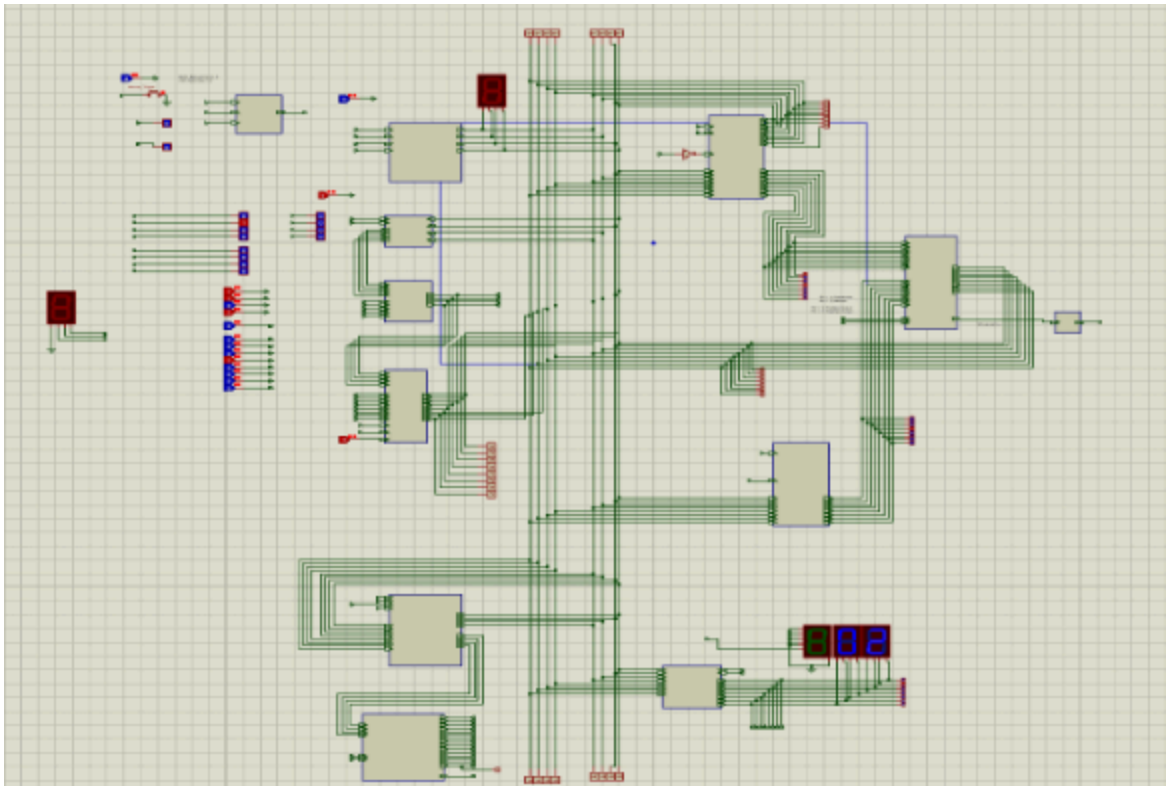
Circuit Connection:

The bus is an 8-bit wide parallel connection running throughout the SAP-1 system, linking the Program Counter, Memory Address Register (MAR), RAM, Instruction Register (IR), Accumulator (A), Register B, ALU, and Output Register. Each module capable of placing data onto the bus is connected through a tri-state buffer, ensuring controlled access. The enable and direction control signals of these buffers are driven by the Controller Sequencer, which activates the appropriate output enable signal for exactly one module at a time. Modules that only receive data, such as Register B and the Output Register, are connected directly to the bus through input latches and do not require tri-state output drivers. This arrangement prevents bus contention and ensures reliable data transfer across the system.

Working Principle:

During operation, the Controller Sequencer manages all bus activity by generating control signals that determine which module drives the bus and which module receives data. When the Program Counter output enable signal is asserted, its 4-bit address is placed onto the lower lines of the bus and is latched into the MAR for instruction addressing. When the RAM output enable signal is activated, the instruction or data stored at the address specified by the MAR is placed onto the full 8-bit bus, allowing it to be loaded into the Instruction Register or Accumulator. During arithmetic operations, the ALU output is enabled onto the bus and subsequently loaded into the Accumulator. For output operations, the Accumulator places its value

onto the bus, which is then latched by the Output Register for display. At every stage, only one output enable signal is active, ensuring that no two modules attempt to drive the bus simultaneously. This controlled, sequential sharing of the bus enables efficient communication between all components and ensures correct operation of the SAP-1 system.



Complete SAP-1 Architecture Implementation showing all interconnected modules communicating over the shared 8-bit System Bus

Operations Of SAP-01

We can do summation and subtraction using our SAP-01. There are different OP codes for different operations, which makes them look different.

Method of doing operations: At first, we need to change the mode to program mode. We will give the inputs according to the addresses and save it in our RAM. After saving it we will turn it into our RUN mode. During this mode, we must ensure that we reset our program counter. So that it starts counting from 0 to F and starts doing our operations accordingly.

For summation, we need to use 0001 as our MSB for the address we are going to utilize. And for subtraction, we will use 0010 as MSB before that certain address. However, we need to stop the clock pulse by HLT by making it 1111 0000 for the last run.

It is necessary to clear the garbage values before doing another operation. Otherwise, the minus sign detector won't work properly. As we know, flip-flops hold memory of its previous operation, we might get wrong values from it. So it is necessary to clear the value of the flip-flop.

Based on the provided schematic, the bus integration acts as the main communication highway for the entire computer, represented by the long vertical green wires running down the center of the circuit. Instead of wiring every single component directly to one another, all the major blocks such as the Program counter, RAM, Accumulator, ALU, and various register connect to this single shared 8-bit pathway. To prevent data from crashing together and causing a short circuit, the Controller sequence block at the bottom acts like a traffic cop. During each beat of the Clock pulse (shown in the top left), the controller sends out specific control signals to ensure that only one module is allowed to push its data onto the bus at a time, while telling another specific module to catch and save that data. By keeping all other components disconnected or quiet during this transfer, the bus allows the

computer to safely move instructions and numbers step-by-step to execute a program.

Suppose we want to carry out the operation 5+2-3,

Step 1: Putting in the First Number (LDA 9H)

The Program Counter points to memory address 0H, which is where the instruction LDA 9H (Binary: 0000 1001) is stored.

This instruction tells the computer to load the data from memory address 9H into the Accumulator.

In the data section, address 9H has the value 05H (Binary: 0000 0101). The Accumulator stores this value of 5 as it moves across the W-bus.

Step 2: Add the Second Number (ADD AH)

The Program Counter goes to address 1H and gets the instruction ADD AH (Binary: 0001 1010).

This command tells the computer to get the data at address AH and add it to the value that is already in the Accumulator.

The value at address AH is 02H (in binary, 0000 0010). The B Register gets this value of 2, and the ALU adds it to the 5 that is already in the Accumulator. The Accumulator now has the new total, which is 7.

Step 3: Take away the third number (SUB BH)

The computer gets SUB BH (Binary: 0010 1011) at address 2H. This tells the system to get the data from address BH and take it away from the Accumulator's current total.

Address BH has the value 03H (in binary: 0000 0011). The ALU takes the 7 from the Accumulator, takes away the 3, and gets 4. The Accumulator saves this last result.

Step 4: Getting the Result (OUT)

The instruction at address 3H is OUT (Binary: 0011 0000).

This command makes the Accumulator send its stored value (which is now 4) out onto the W-bus that everyone can see. At the same time, the Output Register is told to read the bus and save the result so that it can be shown to the user.

Step 5: Ending the Program (HLT)

The computer finally sees the HLT instruction (Binary: 1111 0000) at address 0100 (Binary: 4H).

This stops the computer's clock pulses completely, which means the program is done running.

Additional Features:
Doing more than one arithmetic operations in a row: This system can solve longer math problems, such as 5 + 2 - 3. It does this by using the Accumulator. For instance, it saves the answer (7) back into that box after it adds 5 + 2. After that, the computer just takes the next number (3) and subtracts it from the total it saved to get 4. If you have enough memory on your computer, you can keep adding and subtracting like this.
Showing Negative Numbers: We added a separate minus sign light to make the final answers much easier to read. A special memory switch, called a Dual J-K flip-flop, is wired directly into the computer's calculator part (the ALU) and controls this light. The calculator sends a signal to this switch if you do a math problem that goes below zero. The switch takes that signal and turns on the minus sign on the screen. This lets us know whether our results are negative or not.

We have also used an extra program counter in our RAM, which helps us to set all the input of RAM memory to zero. As a result, there won't be any garbage value at the beginning of any operation.

Challenges

1. Clock & Timing Issues: It was challenging to synchronize every component to a single clock signal.

2. Bus Contention: A 8-bit bus is used in SAP-1. Bus contention is a relatively frequent issue in Proteus that occurs when two components attempt to operate the bus concurrently because of improper control signals.
3. Ring Counter : Building a reliable 5 step ring counter (T1–T5) using flip-flops in Proteus requires careful reset logic. An incorrect reset can cause the counter to get stuck or skip states.

4. Output Register & Display: Connecting the output register to a 7-segment display for decimal output required a BCD to 7-segment decoder which added complexity.

5. Debugging Difficulty: There isn't an integrated microarchitecture debugger in Proteus. It takes time to manually examine each signal using virtual tools like oscilloscopes and logic analyzers.

Credits

Phase A	
Clockpulse	Rafi, Samiha
Program Counter	Raad, Nazifa
Phase B	
Ram, Input Unit, Address Selector	Rafin, Raad, Sanajid
Accumulator, B register	Rafi
Phase C	
Controller sequencer	Raad
ALU	Rafin
Phase D	
Output Register	Samiha
Bus integration	Rafi, Rafin, Sanajid, Raad, Nazifa

Video Demonstration: **Untitled design.mp4**